

Safety Barrier Certificates for Path Integral Control: Safety-Critical Control of Quadrotors

Tao Jin , Jian Di , Xinghu Wang , *Member, IEEE*, and Haibo Ji 

Abstract—The safety issue arises as one of the most important requirements for autonomous quadrotor flight. Recently, control barrier function (CBF) technique has been developed to provide necessary and sufficient conditions on safety for robot systems. Previous works unified CBF with control techniques in an optimization fashion. However, these methods could not provide consistent performance, especially in clutter environments. In this letter, we present a novel predictive control algorithm that combines CBF and path integral control to develop a safety-critical controller for quadrotors. An efficient sequential projection algorithm is adopted to steer unsafe sampled inputs to a feasible region limited by safety barrier certificates. Comparing to existing methods, one distinguishing feature of our proposed method is that we provide collective thrusts and body rates (CTBR) control policies. Simulations and experiments demonstrate that our controller can deliver more robust maneuvers in the course of obstacle avoidance than state-of-the-art safe controllers.

Index Terms—Robot safety, aerial systems: mechanics and control, model predictive path integral control, control barrier function, collision avoidance.

I. INTRODUCTION

A. Motivation

THE last decades have witnessed the rapid development of the quadrotor. Nowadays, the quadrotor has achieved widespread application in a variety of scenarios, such as aerial photography, crop dusting, inspections, and so on. Consequently, the safety issue of the quadrotor has received increasing attention. There are numerous methods that can be employed to realize safety-critical control, such as potential field technique [1], velocity obstacle technique [2], geometric technique [3], reinforcement learning technique [4], etc.

Recently, CBF [5] has received widespread development to ensure the safety properties of various robot systems. The notion of CBF is motivated by control Lyapunov function (CLF) which demonstrates the asymptotically stabilizability of the system

Manuscript received 21 June 2023; accepted 27 July 2023. Date of publication 2 August 2023; date of current version 11 August 2023. This letter was recommended for publication by Associate Editor Francesco Pierri and Editor Clement Gosselin upon evaluation of the reviewers' comments. This work was supported by the National Natural Science Foundation of China under Grants 61873250 and 61873029. (Corresponding author: Xinghu Wang.)

The authors are with the Department of Automation, University of Science and Technology of China, Anhui Province 230026, China (e-mail: lucas@mail.ustc.edu.cn; dijian@mail.ustc.edu.cn; xinghuw@ustc.edu.cn; jihb@ustc.edu.cn).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2023.3301297>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2023.3301297

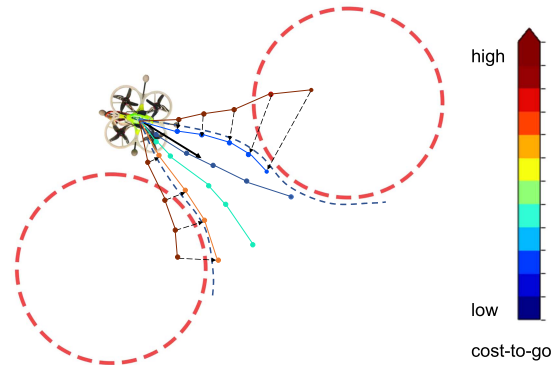


Fig. 1. Illustration of our safety barrier certificates. At each time step, random trajectories [lines with markers] are sampled by Monte Carlo method in MPPI algorithm. Collisions resulting from obstacles [red dashed lines] lead to high cost-to-go, which makes trajectories invalid. Safety barrier certificates steer unsafe inputs into safety region which guarantee the sampling efficiency of MPPI.

under suitable inputs. Essentially, it provides a rigorous characterization of the space of inputs that ensures the forward invariance of a safety set.

To synthesize a safety-critical controller, CBF is first unified with CLF via quadratic programming (QP) which minimizes modifying stabilizing inputs to guarantee safety. The latest work [6] introduces CBF as additional hard constraints in the model predictive control (MPC) problem. Nevertheless, it is challenging for optimization solvers to obtain optimal control policies in real time since complex system dynamics and additional safety requirements naturally constitute nonlinear constraints in the optimization problem.

While optimization based methods suffer from insurmountable challenges, the emerging path integral control method provides insight of designing controllers using Monte Carlo sampling, which inherently benefits us with gradient-free optimization. Model predictive path integral control (MPPI) [7] further develops a feasible iteratively update control law. All these techniques can largely reduce the computation burden and thus make it an achievable method to develop a safety-critical controller [8], [9]. However, the existence of obstacles could make sample inputs invalid. Excessive invalid sample inputs will lead to terrible degradation of sampling efficiency; see Fig. 1 for an intuitive illustration.

Moreover, existing safety works [10] only consider the kinematic dynamics of quadrotors. The neglect of inner loop dynamics gives rise to the possibility of collisions. An experiment comparison [11] demonstrates that control policies producing

collective thrusts and body rates outperform other control policies in dynamics mismatch and high agility. However, the problem of designing a safety-critical controller that takes quadrotor dynamics into consideration, has been rarely studied.

In this letter, we present a safety barrier certificates enhanced MPPI scheme for safety-critical control of the quadrotor. CBF is utilized to develop safe barrier certificates, equivalently converting position based safety constraints into input constraints. A sequential projection algorithm is designed to steer sampled inputs into the feasible region via successive projections. These safe sampled inputs are exploited to obtain optimal CTBR control policies via path integral method. Comparing to previous works, our proposed method builds up the stabilization and robustness of the safety-critical controller using the methods of CBF, MPPI and CTBR control policies.

B. Contribution

We summarize the contributions of this letter as follows.

- i) We incorporate safety barrier certificates into the MPPI control framework via a computation-efficient sequential projection algorithm to realize a real-time controller of the quadrotor.
- ii) To the best of our knowledge, we are the first to develop a safety-critical controller producing CTBR control policies, which guarantee the safety of the quadrotor at the angular velocity level.
- iii) We conduct both simulations and laboratory experiments to validate the superiority of our method in comparison with state-of-the-art methods.

C. Related Works

CBF provides theoretical safety certificates for nonlinear control affine systems. In [5], CBF is unified with CLF to formulate a QP problem that minimizes modifying inputs to guarantee the safety of the system. Considering the nonlinear underactuated dynamics of the quadrotor, a CBF based cascade controller is presented in [12] to ensure the safety of the quadrotor. In [13], a comparison analysis of CBF and artificial potential fields is carried out to show the superiority of CBF.

Safety-critical control can also be resolved by MPC methods. In [14] the authors present an adaptive MPC controller to avoid collision, where extra safe margins are adopted to address the model parameter uncertainty of the quadrotor. A nonlinear MPC method is proposed in [15], with both considering dynamic and geometric constraints in the environment. In [16], a distributed MPC method is presented to generate safe trajectories for the quadrotor swarm in real time. The proximal averaged Newton algorithm is utilized to accelerate the computation process in [17], in which the quadrotor successfully avoids being smashed by moving obstacles.

MPPI based navigation method is first presented in [8], where safety constraints are directly incorporated in the cost function. The covariance steering technique is introduced in [18] to improve the performance of MPPI, where the enhanced controller outperforms the original one in car racing. A CBF augmentation MPPI method is first adopted in [19], where the trust region of

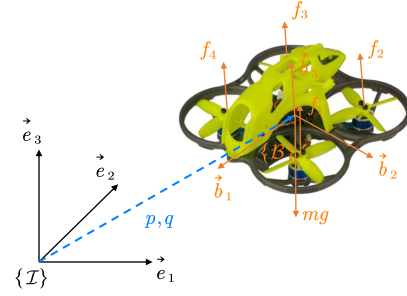


Fig. 2. Motion description of the quadrotor which can be described with the inertial frame $\{I\}$ and the body frame $\{B\}$.

inputs is calculated by stochastic QP to guarantee the safety of a two-dimensional unicycle model. It should be noted that our work differs from theirs in the following ways: *i)* their work requires repeatedly solving semi-definite programming problems at each time step, making it unpractical for real-time computing. *ii)* Our work expands the CBF based MPPI method to a more general system with inputs appearing in derivatives of CBF of different orders. *iii)* We manage to implement our proposed algorithm to the quadrotor system and contrast experiments are conducted to illustrate its performance.

D. Outline

The rest of the letter is organized as follows: In Section II, we present the system dynamics of the quadrotor and the concerned problem formulation. In Section III, we first revisit theories of MPPI and CBF and then introduce our proposed safety-critical controller. In Section IV, we present the results and analysis of simulations and real world experiments. In Section V, we conclude a brief summary of our work.

II. PROBLEM FORMULATION

In this section, we present the dynamic model of the quadrotor and provide a brief description of the safety-critical problem.

As shown in Fig. 2, let p and v denote the position and the velocity of the body frame $\{B\}$ with respect to the inertial frame $\{I\}$. We use the unit quaternion q to represent the rotation of the body frame. Let $T = \frac{1}{m} \sum_{i=1}^4 f_i$ denote the mass m normalized collective thrust and $\omega = [\omega_x, \omega_y, \omega_z]^T$ denote the body rates of the body frame. Then the quadrotor dynamics can be given as follows:

$$\begin{aligned} \dot{p} &= v \\ \dot{v} &= g_z + q \otimes T \otimes q^* \\ \dot{q} &= \frac{1}{2} q \otimes \omega \end{aligned} \quad (1)$$

where g_z denotes the gravity vector and the symbol $\otimes$ represents the quaternion product. Denote that $x = [p^T, v^T, q^T]^T$, $u = [T, \omega^T]^T$. We also consider that there exists external control dependent noise in the system. The above quadrotor dynamics can be reformulated in the following compact form

$$dx = f(x)dt + g(x)(udt + dW) \quad (2)$$

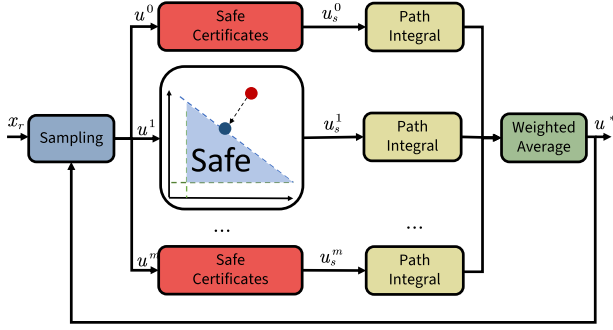


Fig. 3. Brief illustration of our proposed algorithm structure.

where $f(t)$, $g(x)$ can be determined directly by equation (1) and dW is a Wiener process with zero mean and Σ variance.

The objective of our algorithm is to find optimal control policies that satisfy safety constraints for all time. Without loss of generality, we define the free space set $\mathcal{C}$ as

$$\mathcal{C} = \bigcap_{i=1}^{N_o} \mathcal{C}_i, \quad \mathcal{C}_i = \{x \in \mathbb{R}^n | h_i(x) \geq 0\} \quad (3)$$

where N_o denotes the number of obstacles and the continuously differentiable function $h_i(x)$ denotes the safety region except for the i th obstacle. The problem statement is how to design control policies such that the quadrotor can track the desired trajectory while avoiding all possible collisions and achieve certain minimum cost simultaneously. For this purpose, we formulate this problem as a finite horizon optimal control problem of the stochastic quadrotor system

$$V(x) = \min_u \mathbb{E}_{\mathbb{Q}} \left[\Phi(x_{t_f}) + \int_0^{t_f} \left(Q(x) + \frac{1}{2} u^T R u \right) dt \right] \quad (4)$$

$$\text{s.t. } x(t) \in \mathcal{C}, \quad \forall t \in [0, t_f] \quad (5)$$

where expectations $\mathbb{E}_{\mathbb{Q}}[\cdot]$ are taken with respect to (2), $\Phi(x_{t_f})$ denotes the terminal cost, $Q(x)$ denotes the state cost, R denotes the weight matrix of the control cost, and t_f denotes the time horizon.

III. METHODOLOGY

A. Algorithm Structure

As shown in Fig. 3, we present the CBF based MPPI algorithm to address the safety-critical control problem of the quadrotor. At each time step, new control input sequences are sampled based on the last optimized input sequence. Then the sequential projection algorithm based on safety barrier certificates is designed to steer unsafe inputs into the safe region. The optimal input sequence is estimated as the weighted average of sampled control sequences and only the first input is applied to the real plant.

B. Model Predictive Path Integral Control

The fundamental idea of MPPI [20] is to transform the stochastic optimal control problem into an expectation over all possible trajectories using the Feynman-Kac formula. The expectation can be further approximated by forward sampling of stochastic dynamic diffusion processes.

To derive the optimal control method, we first formulate the associated stochastic Hamilton-Jacobi-Bellman (HJB) equation as follows:

$$-\partial_t V = Q(x) + f^T(x) V_x - \frac{1}{2} V_x^T g(x) R^{-1} g^T(x) V_x + \frac{1}{2} \text{tr} (g(x) \Sigma g^T(x) V_{xx}) \quad (6)$$

where V_x and V_{xx} denote first-order and second-order partial derivatives of $V(x)$ with respect to the state x and the corresponding optimal control u is formulated as

$$u^* = -R^{-1} g^T(x) V_x \quad (7)$$

To solve the above partial differential equation (PDE), we take the exponential of the value function $V(x) = -\lambda \log(\Psi(x))$. Besides assuming that $\lambda R^{-1} = \Sigma$, the HJB equation can be re-expressed in the form of a linear partial differential equation:

$$\partial_t \Psi = \frac{\Psi(x)}{\lambda} Q(x) - f^T(x) \Psi_x - \frac{1}{2} \text{tr} (g(x) \Sigma g^T(x) \Psi_{xx}) \quad (8)$$

with the boundary condition $\Psi(x_{t_f}) = \exp(-\frac{1}{\lambda} \Phi(x_{t_f}))$. Applying the Feynman-Kac formula, the PDE can be solved by simulating random trajectories of a stochastic process. Meanwhile we introduce the important sampling method [21] to improve sample efficiency, which leads to the following iterative update law:

$$u_k^* = u_k + \frac{\sum_{m=1}^M \exp\left(-\frac{1}{\lambda} \tilde{S}(\tau^m)\right) \delta u_k^m}{\sum_{m=1}^M \exp\left(-\frac{1}{\lambda} \tilde{S}(\tau^m)\right)} \quad (9)$$

$k = 0, 1, \dots, N-1$

where N denotes the number of time steps, M denotes the number of rollouts, δu denotes the virtual input disturbance derived from noise, and $\tilde{S}(\tau)$ is the modified cost-to-go defined as

$$\tilde{S}(\tau) = \Phi(x_{t_f}) + \sum_{k=0}^{N-1} \tilde{Q}(x_k, u_k, \delta u_k) \quad (10)$$

$$\tilde{Q}(x, u, \delta u) = Q(x) + \frac{1 - \nu^{-1}}{2} \delta u^T R \delta u + u^T R \delta u + \frac{1}{2} u^T R u \quad (11)$$

where ν determines the degree of exploration of the state space.

C. Control Barrier Function

In order to simplify the problem, we encapsulate obstacles into ellipsoids to describe unsafe regions, such that $h_i(x)$ can be

expressed as

$$h_i(x) = \|p - p_i^o\|_{Q_p}^2 - r_i^2 \quad (12)$$

where Q_p denotes the shape of the ellipsoid, p_i^o and r_i denote the center position and the inflated radius of the i th obstacle respectively.

In previous works, CBF was introduced to develop state dependent linear in-equality constraints for safety control, which requires that inputs appear explicitly in the same derivative of CBF. In our case, the thrust appears in the third derivative of $h_i(x)$ while body rates appear in the fourth derivative, which does not conform with the condition of CBF. To overcome this hurdle, we introduce a new virtual input ζ to be the difference of collective thrusts T such that this newly introduced virtual input together with body rates appears in the fourth derivative

$$\zeta_k = \frac{T_{k+1} - T_k}{\Delta t} \quad (13)$$

Substituting (13) into (2) with $dW = 0$ yields new system dynamics

$$\bar{x}_{k+1} = \bar{x}_k + (\bar{f}(\bar{x}_k) + \bar{g}(\bar{x}_k)\bar{u}_k) \Delta t \quad (14)$$

where $\bar{x}_k = [x_k^T, T_k]^T$ and $\bar{u}_k = [\zeta_k, \omega_k^T]^T$. Then we can introduce a set of functions

$$h_i^0(\bar{x}_k) = h_i(\bar{x}_k) \quad (15)$$

$$h_i^1(\bar{x}_k) = h_i^0(\bar{x}_{k+1}) - h_i^0(\bar{x}_k) + c_1 h_i^0(\bar{x}_k) \quad (16)$$

$$h_i^2(\bar{x}_k) = h_i^1(\bar{x}_{k+1}) - h_i^1(\bar{x}_k) + c_2 h_i^1(\bar{x}_k) \quad (17)$$

$$h_i^3(\bar{x}_k, \bar{u}_k) = h_i^2(\bar{x}_{k+1}) - h_i^2(\bar{x}_k) + c_3 h_i^2(\bar{x}_k) \quad (18)$$

where c_1, c_2 and c_3 are positive coefficients of CBF. As stated in [22], consider the system (14) and safety set $\mathcal{C}$, $h_i(\bar{x}_k)$ is a discrete exponential CBF if the initial state $\bar{x}_0$ stays in the safety set, which equals the following constraints:

$$h_i^j(\bar{x}_0) \geq 0, \quad j = 0, 1, 2 \quad (19)$$

and

$$h_i^3(\bar{x}_k, \bar{u}_k) \geq 0, \quad k = 1, 2, \dots, N-1 \quad (20)$$

Following this, we have $h_i(\bar{x}_k) \geq 0, \forall k = 0, 1, \dots, N-1$, i.e., the quadrotor will not collide with the i th obstacle.

D. Sequential Projection Algorithm

CBF converts safety constraints that are explicitly based on position into state-related input constraints. In this manner, we can constrain new disturbed control sequences before the path integral stage. This can significantly improve sampling efficiency. To guarantee sampled inputs lie in the safety region, CBF based input constraints should be satisfied for all $i = 0, 1, \dots, N_o - 1$. Note that all inputs are affine in quadrotor dynamics, thus the inequality (20) can be re-expressed in the following form:

$$A_{h_i}(\bar{x})\bar{u} + b_{h_i}(\bar{x}) \geq 0, \quad i = 0, 1, \dots, N_o - 1 \quad (21)$$

These linear inequalities define half-spaces in $\mathbb{R}^4$. For the inequality that does not hold, we can project inputs onto the

Algorithm 1: Sequential Projection Algorithm.

Require: x_k : Initial state of current time step;

u_k, u_{k-1} : Input of current time step and last time step;

u_{lb}, u_{ub} : Input boundary;

$h_i(x), i = 0, 1, \dots, N_o$: Candidate CBFs;

for $j \leftarrow 0$ to $N_{it} - 1$ **do**

$v_k \leftarrow \frac{T_k - T_{k-1}}{\Delta t}$

for $i \leftarrow 0$ to $N_o - 1$ **do**

$\bar{u}_k \leftarrow \mathcal{P}_{h_i}(\bar{u}_k)$

end for

$T_k \leftarrow v_k \Delta t + T_{k-1}$

$u_k \leftarrow \text{SAT}(u_k)$

end for

corresponding hyper-plane to make it feasible. This yields the following equation:

$$\mathcal{P}_{h_i}(\bar{u}) = \begin{cases} \bar{u}, & \text{if (21) holds} \\ \bar{u} - \frac{A_{h_i}\bar{u} + b_{h_i}}{\|A_{h_i}\|} A_{h_i}^T, & \text{else} \end{cases} \quad (22)$$

Besides, we consider the boundary constraints of inputs, which can be realized by the saturation function

$$\text{SAT}(u) = \begin{cases} u_{ub}, & u > u_{ub} \\ u, & u_{lb} \leq u \leq u_{ub} \\ u_{lb}, & u < u_{lb} \end{cases} \quad (23)$$

where u_{lb} and u_{ub} denote the lower and upper bound of control inputs respectively.

Our objective is to find corresponding valid inputs lying at the intersection of half-spaces. For this purpose, we successively project infeasible inputs to approach the intersection. It is easy to see that every projection could make the input closer to the intersection. Since the distance is bounded, the input will ultimately converge to a certain point in the intersection, which illustrates the feasibility of the sequential projection algorithm. In practice, to adapt the algorithm to the parallel computation fashion implemented in the graphics processing unit, we set up a maximum iteration number N_{it} such that all threads can terminate at the same time. This practice can guarantee inputs in the feasible region as much as possible while ensuring computational efficiency. We summarize the sequential projection algorithm in Algorithm 1.

E. Summary

We present our proposed algorithm in Algorithm 2, named SBCPI in the following sections. In practice, we utilize the output collective thrust of previous time step to calculate the virtual input. The distinguishing feature of our algorithm is that the difference (13) can be calculated directly utilizing the sampled inputs of MPPI. Therefore, the next state can be numerically integrated using the current state and input, while the next safe input can be constrained by using the next state alternately. Moreover, these calculations do not require to compute derivatives and can be completed in finite steps in a parallel fashion. All of these treatments above make our algorithm more efficient than all previous optimization based algorithms.

Algorithm 2: SBCPI Algorithm.

Require: $f, g, \Delta t$: Discrete dynamics;
 u_{lb}, u_{ub} : Inputs boundary;
 Q, Φ, R, λ, ν : MPPI functions and parameters;
 $h_i(x), c_1, c_2, c_3$: CBFs and parameters;
 M, N : Number of rollouts and time steps;
 x_0 : Initial state of the quadrotor;
while not finished **do**
 for $m \leftarrow 0$ to $M - 1$ **do**
 $x_0^m \leftarrow x_0$
 Sample δu^m
 $u^m \leftarrow [u_0 + \delta u_0^m, u_1 + \delta u_1^m, \dots, u_{N-1} + \delta u_{N-1}^m]$
 for $k \leftarrow 0$ to $N - 1$ **do**
 $u_k^m \leftarrow$
 Sequential_Projection_Algorithm(x_k^m, u_k^m)
 $x_{k+1}^m \leftarrow x_k^m + (f + gu_k^m)\Delta t$
 $\tilde{S}(\tau^m) \leftarrow \tilde{S}(\tau^m) + \tilde{Q}(x_k^m, u_k, \delta u_k^m)$
 end for
 end for
 for $k \leftarrow 0$ to $N - 1$ **do**
 $u_k^* \leftarrow u_k + \frac{\sum_{m=1}^M \exp(-\frac{1}{\lambda} \tilde{S}(\tau^m)) \delta u_k^m}{\sum_{m=1}^M \exp(-\frac{1}{\lambda} \tilde{S}(\tau^m))}$
 end for
 send to the quadrotor(u_0^*)
end while

TABLE I
CONTROLLER PARAMETERS

Name	Value	Name	Value
$\omega_{\max}$	1.0	λ	0.08
$T_{\max}$	12.0	Σ	diag[0.4, 0.4, 0.4]
Δt	0.05	ν	1000
M	10240	c_1	3.0
N	20	c_2	5.0
N_{it}	10	c_3	5.0

IV. RESULTS

In this section, we design both simulations and real world experiments to illustrate the performance of our proposed algorithm. We also compare the proposed algorithm to state-of-the-art algorithms in their trajectory tracking and collision avoidance performance.

A. Controller Setup

In order to achieve the goal of tracking desired trajectories and avoiding collisions, the cost function $Q(x)$ here consists of two parts: a cost for deviating from the trajectory and a penalty for collisions between the quadrotor and obstacles

$$\Phi(x) = \|x - x_r\|_2 + 10^5 \cdot \mathcal{L} \left(- \sum_{i=0}^{N_o-1} h_i(x) \right) \quad (24)$$

where x_r denotes the reference state and $\mathcal{L}(\cdot)$ denotes the indicator function, which is designed to filter remaining infeasible inputs. Controller parameters are tabulated in Table I. The proposed SBCPI controller is implemented in C++ and the parallel

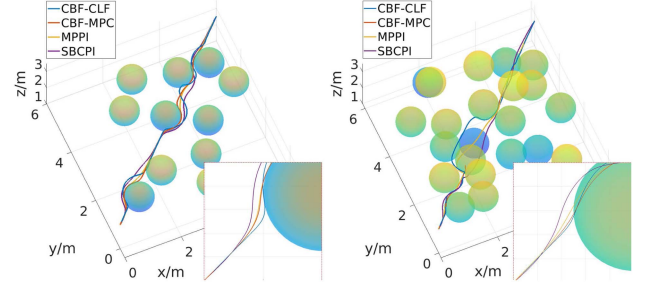


Fig. 4. Perspective view of simulation results (left: Map I; right: Map II). The lines with different colors stand for the trajectories of the quadrotor under the control of different controllers. The bottom right figures are the partial top view where the quadrotor is approaching the first obstacle.

computing is accelerated by CUDA. The whole algorithm is set up to run at a frequency of 120 Hz.

B. Contrast Controller Setup

To demonstrate the superiority of our proposed controller, we design contrast experiments to compare it with three state-of-the-art safety controllers. To reduce the influence of irrelevant variables, we first implement a full PID inner-outer loop controller as proposed in [23] as the standard controller. Then the counterpart of the controller is modified to realize our controller and contrast controllers. Contrast controllers are listed as follows:

1) CBF-CLF refers to the QP based controller combining CBF and CLF as proposed in [5]. The controller outputs safety velocity commands for an attitude controller to track. The numerical optimization package OSQP [24] is employed to solve the QP problem.

2) CBF-MPC refers to the CBF based MPC controller as it is proposed in [17]. The CBF-MPC controller computes desired attitude control policies. The MPC controller is set up with ACADO [25] and the optimization problem is solved by qpOASES [26].

3) MPPI refers to the standard MPPI controller. All the setups are kept identical with our proposed controller, except that we remove the safety barrier certificates.

C. Simulation

1) *Scenario Setup*: To benchmark the four controllers mentioned above, we establish a simulation platform based on Gazebo and the open-source package RPG Quadrotor Control [27]. Spheres with a radius of 0.5 m are randomly arranged in 6 m * 6 m * 6 m space to simulate obstacles in the real world. The goal of the quadrotor is to move across the space full of obstacles without colliding. We assume that the quadrotor has no information about obstacles at the planning stage, thus the reference trajectory is straight-forward from start to finish. Controllers are supposed to guide the quadrotor to move towards the destination and avoid possible collisions. We randomly generate two maps named Map I and Map II as shown in Fig. 4.

2) *Result Analysis*: According to Fig. 4, we can see that those model predictive based controllers could drive the quadrotor

TABLE II
DISTANCE TO OBSTACLES

Distance	Map I		Map II	
	average	minimum	average	minimum
CBF-CLF	0.36 m	-0.03 m	0.31 m	-0.04 m
CBF-MPC	0.36 m	0.04 m	NaN	NaN
MPPI	0.37 m	-0.002 m	0.31 m	-0.001 m
SBCPI	0.41 m	0.06 m	0.37 m	0.04 m

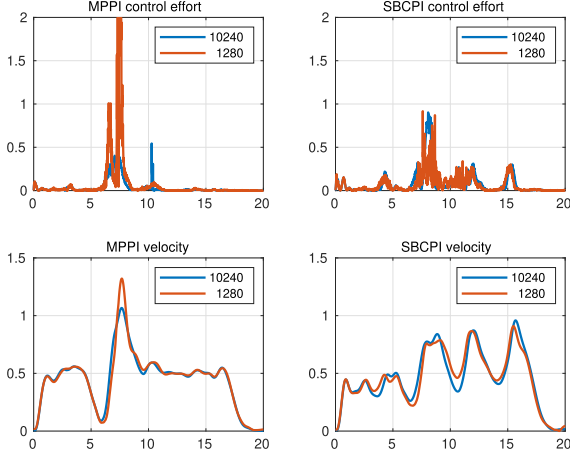


Fig. 5. Control effort and velocity of MPPI and SBCPI controllers with different numbers of sampling, which is indicated in the legend. Control effort is defined as $\|u - u_h\|_2$, where u_h denotes the input in the hovering state.

to avoid obstacles earlier than the CBF-CLF controller thanks to their nature of predictive control. Moreover, facing more obstacles in Map II, the quadrotor is unexpectedly blocked by the first few obstacles under the control of the CBF-MPC controller. This is caused by hard safety constraints in the optimization problem, which make the solver encounter insoluble situations in limited computing time. We also undertake investigations into the distance to obstacles of all trajectories, which is concluded in Table II. We notice that although all the controllers can make the quadrotor cross the area with obstacles, contrast controllers cannot strictly comply with predefined safety distances. This circumstance could result from the imperfect tracking performance of the inner loop controller. On the other hand, since CBF can introduce additional safety margins when the quadrotor is getting close to the obstacle, the SBCPI controller can keep proper distance to obstacles comparing to the MPPI controller.

Next, we make a more specific comparison between the MPPI controller and the SBCPI controller. Except for the above experiments, we decrease the number of samples to 1280 to study its influence on the controller performance. Fig. 5 shows the control effort and velocity of the quadrotor under the control of different controllers. We can see that both controllers are reliable when sampling is adequate. However, the output control policies of MPPI become noisy and unstable when the number of samples is decreased. There is a severe fluctuation when the quadrotor meets dense obstacles, which directly leads to a peak velocity in the curve. On the other hand, the SBCPI controller maintains almost identical performance in both cases. This illustrates that the existing obstacles could greatly influence the performance

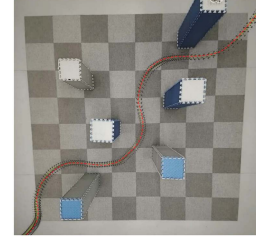


Fig. 6. Top view of the experiment scenario.

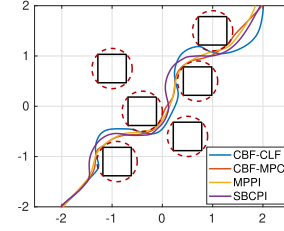


Fig. 7. Top view of trajectories [black squares denote obstacles and red dots denote adopted circumcircles].

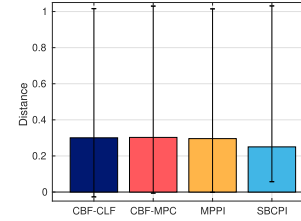


Fig. 8. Comparison of distance to obstacles of four trajectories.

of MPPI. The additional safety certificates certainly help to improve the sampling efficiency.

D. Real World Experiment

1) *Scenario Setup*: We modify a LDARC ET85 FPV drone as our experiment quadrotor, which features an 85 mm wheelbase and a thrust-to-mass ratio of around 3.3. The OptiTrack motion capture system is utilized to acquire the real time pose of the quadrotor. The algorithm is deployed on the desktop computer, and control commands are packaged to SBUS signals and sent to the RadioMaster TX16S radio transmitter via the UART interface. The transmitter is configured in slave mode to relay commands to the quadrotor via the TBS Tracer 2.4 GHz low-latency radio link. To validate the obstacle avoidance performance of controller, we place five cuboid obstacles in the field. Like the simulation scenario, the stated goal of the quadrotor is to move diagonally across the obstacles, as it is shown in Fig. 6.

2) *Result Analysis*: The results are depicted in Figs. 7, 8, and 9. Different from simulations, there is model uncertainty and disturbance in real world experiments. In this case, only the SBCPI controller does not violate the pre-set safety distance and the average distance to obstacles is slightly lower than that of contrast groups. This validates the conclusion of [11] that

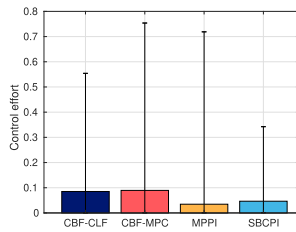


Fig. 9. Comparison of control effort of four controllers.

CTBR control policies could exhibit superior performance. We also compare their performance on control effort as shown in Fig. 9. There is more apparent fluctuation in the control inputs of contrast controllers which is directly reflected in the higher peak of control effort. Whereas the SBCPI controller could maintain consistent performance with a lower control effort variance. Synthesizing above analysis, we can conclude that the SBCPI controller outperforms contrast controllers in terms of collision avoidance.

V. CONCLUSION

In this letter, we propose a sampling-based safety-critical controller for quadrotor flight. The results show that our method provides more reliable and robust performance in collision avoidance. Nevertheless, CBF is only suitable for regular obstacles, which can be described by continuously differentiable functions. How to employ it in complex environments remains to be studied.

REFERENCES

- [1] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Proc. IEEE Int. Conf. Robot. Automat.*, 1985, pp. 500–505.
- [2] J. Snape, J. V. d. Berg, S. J. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle," *IEEE Trans. Robot.*, vol. 27, no. 4, pp. 696–706, Aug. 2011.
- [3] D. Zhou, Z. Wang, S. Bandyopadhyay, and M. Schwager, "Fast, on-line collision avoidance for dynamic vehicles using buffered Voronoi cells," *IEEE Robot. Automat. Lett.*, vol. 2, no. 2, pp. 1047–1054, Apr. 2017.
- [4] L. Wang, E. A. Theodorou, and M. Egerstedt, "Safe learning of quadrotor dynamics using barrier certificates," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 2460–2465.
- [5] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
- [6] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *Proc. Amer. Control Conf.*, 2021, pp. 3882–3889.
- [7] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *J. Guid., Control, Dyn.*, vol. 40, no. 2, pp. 344–357, 2017.
- [8] I. S. Mohamed, G. Allibert, and P. Martinet, "Model predictive path integral control framework for partially observable navigation: A quadrotor case study," in *Proc. Int. Conf. Control, Automat., Robot. Vis.*, 2020, pp. 196–203.
- [9] G. Williams, B. Goldfain, P. Drews, K. Saigol, J. M. Rehg, and E. A. Theodorou, "Robust sampling based model predictive control with sparse objective information," in *Proc. Robot.: Sci. Syst.*, 2018, p. 2018.
- [10] M. Hoy, A. S. Matveev, and A. V. Savkin, "Algorithms for collision-free navigation of mobile robots in complex cluttered environments: A survey," *Robotica*, vol. 33, no. 3, pp. 463–497, 2015.
- [11] E. Kaufmann, L. Bauersfeld, and D. Scaramuzza, "A benchmark comparison of learned control policies for agile quadrotor flight," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 10504–10510.
- [12] M. Khan, M. Zafar, and A. Chatterjee, "Barrier functions in cascaded controller: Safe quadrotor control," in *Proc. Amer. Control Conf.*, 2020, pp. 1737–1742.
- [13] A. Singletary, K. Klingebiel, J. Bourne, A. Browning, P. Tokumaru, and A. Ames, "Comparative analysis of control barrier functions and artificial potential fields for obstacle avoidance," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2021, pp. 8129–8136.
- [14] G. Garimella, M. Shekells, and M. Kobilarov, "Robust obstacle avoidance for aerial platforms using adaptive model predictive control," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 5876–5882.
- [15] S. S. Mansouri et al., "A unified NMPC scheme for MAVs navigation with 3D collision avoidance under position uncertainty," *IEEE Robot. Automat. Lett.*, vol. 5, no. 4, pp. 5740–5747, Oct. 2020.
- [16] C. E. Luis, M. Vukosavljev, and A. P. Schoellig, "Online trajectory generation with distributed model predictive control for multi-robot motion planning," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 604–611, Apr. 2020.
- [17] B. Lindqvist, S. S. Mansouri, A.-A. Agha-mohammadi, and G. Nikolakopoulos, "Nonlinear MPC for collision avoidance and control of UAVs with dynamic obstacles," *IEEE Robot. Automat. Lett.*, vol. 5, no. 4, pp. 6001–6008, Oct. 2020.
- [18] J. Yin, Z. Zhang, E. Theodorou, and P. Tsotras, "Trajectory distribution control for model predictive path integral control using covariance steering," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2022, pp. 1478–1484.
- [19] C. Tao, H. Kim, H. Yoon, N. Hovakimyan, and P. Voulgaris, "Control barrier function augmentation in sampling-based control algorithm for sample efficiency," in *Proc. Amer. Control Conf.*, 2022, pp. 3488–3493.
- [20] E. A. Theodorou, "Nonlinear stochastic control and information theoretic dualities: Connections, interdependencies and thermodynamic interpretations," *Entropy*, vol. 17, no. 5, pp. 3352–3375, 2015.
- [21] G. Williams, A. Aldrich, and E. Theodorou, "Model predictive path integral control using covariance variable importance sampling," 2015, *arXiv:1509.01149*.
- [22] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," in *Proc. Amer. Control Conf.*, 2016, pp. 322–328.
- [23] R. Mahony, V. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor," *IEEE Robot. Automat. Mag.*, vol. 19, no. 3, pp. 20–32, Sep. 2012.
- [24] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Math. Program. Comput.*, vol. 12, no. 4, pp. 637–672, 2020.
- [25] B. Houska, H. Ferreau, and M. Diehl, "ACADO toolkit—an open source framework for automatic control and dynamic optimization," *Optimal Control Appl. Methods*, vol. 32, no. 3, pp. 298–312, 2011.
- [26] H. Ferreau, C. Kirches, A. Potschka, H. Bock, and M. Diehl, "qpOASES: A parametric active-set algorithm for quadratic programming," *Math. Program. Comput.*, vol. 6, no. 4, pp. 327–363, 2014.
- [27] M. Faessler, D. Falanga, and D. Scaramuzza, "Thrust mixing, saturation, and body-rate control for accurate aggressive quadrotor flight," *IEEE Robot. Automat. Lett.*, vol. 2, no. 2, pp. 476–482, Apr. 2017.